

AGE 316: MICROCOMPUTER SYSTEMS

LECTURE 12: TIMERS & NON-BLOCKING CODE

Dr. Kadeghe Fue

Department of Agricultural Engineering

Recap: Lecture 11

📦 **Functions:** Modular blocks of code that perform specific tasks.

📚 **Libraries:** Collections of functions (e.g., `Servo.h`).

🖌️ **Clean Code:** Writing logic that is easy to read and debug.

> **Today:** We tackle the biggest enemy of efficient code: `delay()`.

The Problem with delay()

```
digitalWrite(led, HIGH);
delay(5000); // Wait 5 seconds
digitalWrite(led, LOW);
```

Blocking Code

`delay()` pauses the **entire processor**.

During that 5 seconds, the Arduino is "asleep".

During delay(), the Arduino CANNOT:

- Read a sensor.
- Detect a button press.
- Update a display.

Analogy: The Blocking Chef



Blocking (Delay)

Put a pizza in the oven for 15 minutes.

Stare at the oven for 15 minutes. Do absolutely nothing else.

Ignore the phone ringing. Ignore the fire alarm.

Inefficient!

Analogy: The Multitasking Chef

Non-Blocking (Millis)

Put pizza in oven. Note the time.

Chop vegetables. Check phone. Wash dishes.

Periodically check: *"Has 15 minutes passed yet?"*

Efficient!



The Solution: millis()

`millis()` returns the number of milliseconds since the Arduino board began running.



It Never Stops

It counts up continuously from 0 immediately after power on.



Read Only

You cannot reset it. You can only read it.



Unsigned Long

It stores a huge number. We use `unsigned long` variables to hold it.

The Logic: "Are we there yet?"

Instead of pausing, we check the difference in time.

```
Current Time - Start Time ⩾ Wait Interval
```



If the difference is small, we keep working. If the difference is big enough, we do the task.

Step 1: Variables

To use this technique, we need to remember the last time we did something.

```
// Constants
const long interval = 1000; // Blink every 1 sec

// State Variables
unsigned long previousMillis = 0; // Last time we blinked
int ledState = LOW;
```

Note: We use `unsigned long` because `int` is too small to hold the number of milliseconds the Arduino runs for (overflows in 32 seconds vs 50 days).

Step 2: The Loop Logic

```
void loop() {
  // 1. Check the time
  unsigned long currentMillis = millis();

  // 2. Compare
  if (currentMillis - previousMillis ⩾ interval) {
    // 3. Save the new time!
    previousMillis = currentMillis;

    // 4. Do the task
    if (ledState == LOW) ledState = HIGH;
    else ledState = LOW;
    digitalWrite(ledPin, ledState);
  }
}
```

Why go through the trouble?

Multitasking!

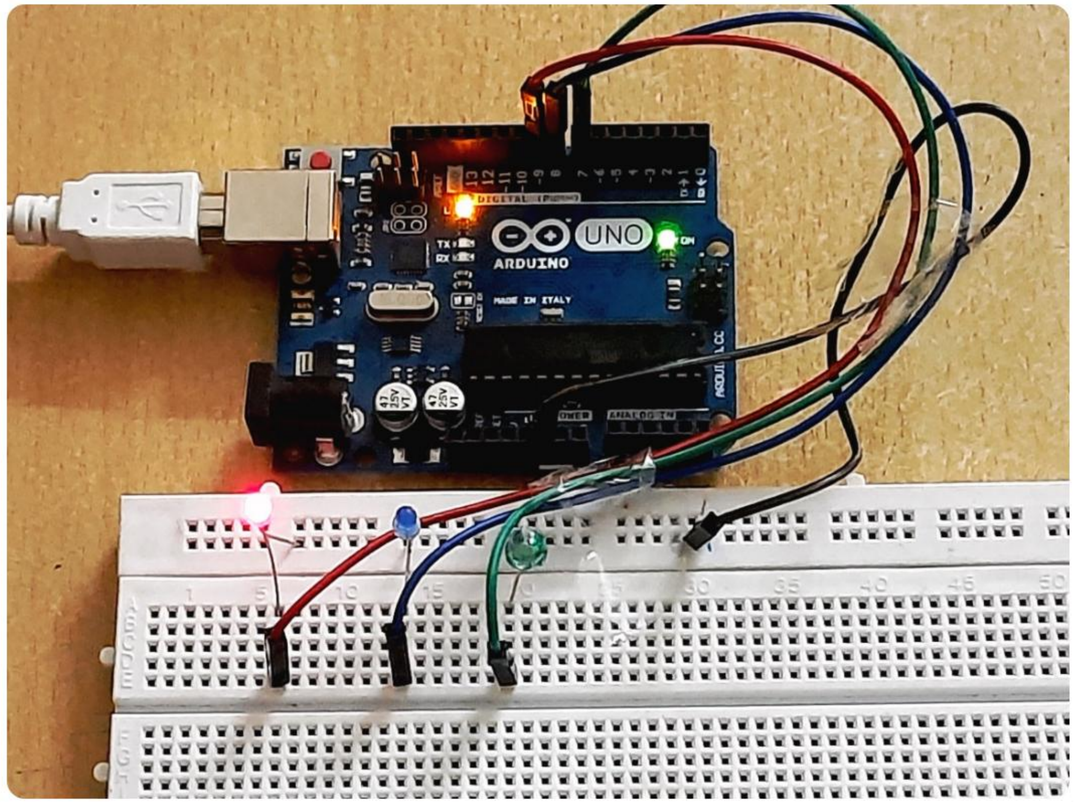
Because the `if` statement only takes a microsecond to check, the processor is free the rest of the time.

Now you can:

- Blink an LED *while* reading a button.
- Update a screen *while* running a motor.



Example: Traffic Light System



Scenario

We want a traffic light to cycle (Red, Green, Yellow) automatically.

BUT we also want a pedestrian button. If pressed, it changes immediately.

With delay(10000) on Green, the button would be ignored for 10 seconds!

With millis(), the button works instantly.

Advanced: State Machines

Using `millis()` often leads to "State Machines".

State 1: IDLE

Waiting for button.

State 2: ACTIVE

Motor running (Timer Start).

State 3: COOLDOWN

Fan On (Timer Check).

We use a `switch...case` variable to track which state we are in.

Multiple Timers

You can run many things at different speeds easily.

```
unsigned long current = millis();

// Timer 1: Fast Blink (LED 1)
if (current - prev1 ≥ 200) {
  prev1 = current;
  // Toggle LED 1
}

// Timer 2: Slow Update (LCD)
if (current - prev2 ≥ 1000) {
  prev2 = current;
  // Update LCD Screen
}
```

Concept: Debouncing

The Bouncing Signal

When you press a button, metal contacts vibrate. The Arduino sees: *High-Low-High-Low-High* in microseconds.

Software Debounce

We use `millis()`! We ignore any signal changes that happen within 50ms of the first press.



Noise

Teaser: Hardware Interrupts

Even `millis()` requires the loop to check repeatedly (Polling).



Interrupt

A hardware signal that immediately pauses the loop, runs a quick function, and returns.



Use Case

Emergency Stops, Precise Encoders.

(We will cover this in advanced lectures if time permits).

Summary



Delay is Bad

It blocks the processor. Use it only for very simple demos.



Millis is Good

Use `millis()` to track time without stopping the code.



Multitasking

By using non-blocking code, we can do multiple things "at once".

Questions?

"The only stupid question is the one you don't ask."

Dr. Kadeghe Fue

Sokoine University of Agriculture

Image Sources



https://www.healthyfood.com/wp-content/uploads/2021/11/How-to-multitask-like-a-chef-iStock-824283794.jpg

Source: www.healthyfood.com



https://img.freepik.com/premium-vector/stopwatch-vector-clipart-illustration-clock-timer-clipart-concept-white-isolated_138676-390.jpg

Source: www.freepik.com



https://projects.arduinocontent.cc/cover-images/8d220b4b-0833-4c7b-8eb7-abe3a8c45996.blob

Source: projecthub.arduino.cc